

Week 3 Plan — SAGE-K8s

Measurement week. Week 2 delivered a working loop; Week 3 must turn it into numbers. Arya (Operator) · Ananya (Safety) · Subhashini (Classifier + Eval). This is the last build week — Week 4 is demo, report and freeze.

Project completion — roughly 60% (loop works end-to-end on a live cluster; zero measured results yet)



- Two findings from Week 2 that reorder this week.
- The missing attempt budget blocks your headline contribution, not just correctness.** A permanently-broken Deployment re-remediated every ~45s forever *is* the unrecoverable arm. With no budget and no terminal "exhausted" state, "one remediation attempt" is undefined — so the rollback-rate denominator is undefined and contribution #1 cannot be computed. This is Day 1 work, before anything else.
 - The null-action baseline is currently designed to measure zero.** Both existing workloads are built so Kubernetes cannot self-heal, so the control arm returns 0/5 and the "net of baseline" contribution says nothing. You need a third workload — a **transient** crash that K8s backoff genuinely recovers — or contribution #2 produces no finding. See Arya Task 3.

- Shared Day 1 decisions (all three, 45 min, before any code).** Write these into the repo as `docs/measurement-definitions.md`. Every metric depends on them and they cannot change after runs begin.
- Incident** = one detection → one terminal outcome. **Attempt** = one snapshot → action → verify cycle. An incident contains 1..N attempts.
 - Terminal outcomes (exactly five):** `recovered`, `rolled_back`, `exhausted` (attempt budget spent), `escalated` (classifier returned `safe_for_automation:false` or unknown), `rejected` (validator refused the proposed action).
 - Rollback trigger rate** — primary denominator is **attempts** (matches the review's "rate over a population of remediation attempts"); report incident-level as secondary. Decide and write both formulas now.
 - TTD** = fault injection → DetectionEvent. **TTM** = DetectionEvent → terminal outcome.
 - Attempt budget:** `MaxAttempts` = 3 per incident, exponential backoff between attempts, cooldown after terminal outcome. Agree the numbers now; they appear in the report.

- Process changes adopted from Week 2 (all three).**
- One in-cluster deployed run per milestone** — Arya is right that two of six bugs were structurally unreachable from `make run`, which uses the developer kubeconfig and never exercises the manager's RBAC. Schedule deployed runs mid-week and end-week.
 - Cross-owner edits need review.** Owner 1 patched the classifier (3 commits, one behavioural) and the safety package (1 commit, behavioural). Ananya and Subhashini: **rebase today** and re-run your own test suites before writing new code. From now on, a cross-package behavioural change needs the owning teammate's approval on the PR — not to slow you down, but because the owner is the one who has to defend that code in the viva.
 - The demo/runbook track is a permanent deliverable**, as Arya said. It feeds Week 4 directly.

Formally cut this week (say it out loud so nobody quietly keeps working on it)

The Ollama / Mistral-7B offline comparison is cut. It was the one item of the four that isn't load-bearing for a contribution claim, and Week 2 added ~800 unplanned lines of evidence collection. It becomes a "future work" line in the report. **Window-sensitivity analysis is also cut** unless Day 5 finishes early.

Arya — Operator Core: Loop Termination, Evidence Integrity, Workloads

Mission: make the loop terminate correctly, stop the controller from poisoning its own evidence, and build the third workload that makes the null-baseline contribution meaningful. You own the three things that block everyone else's measurements.

Day 1 — Attempt budget, backoff, cooldown (blocking everything)

Task 1 — Per-Deployment attempt tracking. Extend the in-flight guard (already keyed by owner Deployment) into an incident record holding: `incidentID` , `attemptCount` , `firstDetectedAt` , `lastAttemptAt` , `terminalOutcome` . One incident per Deployment at a time.

Task 2 — Budget, backoff and cooldown. Three rules: (a) stop after `MaxAttempts = 3` and emit terminal outcome `exhausted` ; (b) exponential backoff between attempts within an incident; (c) after any terminal outcome, a cooldown during which that Deployment is not re-detected — reset early if the Deployment's `generation` changes, since that means a human or a new rollout intervened.

Paste to your AI: "In a controller-runtime Reconcile loop, I track in-flight remediations in a map keyed by owner Deployment UID. Extend this to an incident record with `attemptCount`, `timestamps`, and a terminal outcome. Implement: a max-attempt budget that emits a terminal 'exhausted' outcome; exponential backoff between attempts; and a post-terminal cooldown that is reset early if the Deployment's `metadata.generation` changes. Show me the struct, the guard logic in Reconcile, and explain how you avoid a goroutine leak or a stale entry if the controller restarts mid-incident."

Test this specifically: deploy a permanently-broken Deployment and confirm the controller performs exactly 3 attempts, emits `exhausted` , and then **goes quiet**. If it is still acting at minute 10, the fix is incomplete. This is the single most important test of your week.

Day 2 — Evidence integrity (the 25-event cap problem)

Task 3 — Freeze evidence at detection. You found that remediation's own events crowd the original cause out of the 25-event cap, so attempt 2+ classifies against garbage. Fix: capture the evidence bundle **once, at incident detection**, and reuse that frozen bundle for every attempt in the incident. Additionally filter out events whose source/actor is the SAGE controller itself, so self-generated events can never occupy cap slots.

Paste to your AI: "My controller collects up to 25 Kubernetes Events plus pod logs as classifier evidence. Two problems: (1) the controller's own remediation actions generate Events that crowd out the original failure cause on retry attempts; (2) evidence is re-collected per attempt. Show me how to (a) filter Events by reporting component/source so controller-generated events are excluded, and (b) capture an immutable evidence snapshot at incident detection and reuse it across attempts. Explain the tradeoff of freezing evidence — what do we lose if the true cause only becomes visible after the first attempt?"

Note the tradeoff honestly in your design doc — frozen evidence means a cause that only becomes visible later is missed. That is a defensible choice (it preserves attribution) but it must be stated, not hidden.

Day 2-3 — The third workload (unblocks the null-baseline contribution)

Task 4 — Build W1: the transient crasher. Your two existing workloads are both K8s-unrecoverable by construction, so the control arm measures zero. W1 must be a pod that crash-loops a few times and then **recovers on its own**, so Kubernetes' native backoff genuinely fixes it. Implementation: a container that increments a counter in an `emptyDir` volume (which survives container restarts within the same pod), exits non-zero for the first 3 starts, then runs normally.

Paste to your AI: "Give me a Kubernetes Deployment manifest whose container crashes on its first 3 starts and then runs successfully, using a counter file on an `emptyDir` volume that persists across container restarts within the pod. It must remain in `CrashLoopBackOff` long enough to be detected (so at least 2-3 restarts) and then self-recover without intervention. Explain why `emptyDir` survives container restart but not pod deletion, and what that implies if my controller deletes the pod."

The subtlety that makes this interesting: if your controller deletes the pod, the `emptyDir` counter resets and the crash loop restarts — so `restart_pod` may perform *worse* than doing nothing. That is a real, publishable-quality finding for your report, not a bug. Confirm the behaviour and record it.

Day 4 — Consolidate the workload set

Task 5 — Freeze three named workloads with manifests and runbook entries, extending the demo track you already built:

ID	Workload	Expected without controller	Expected with controller
W1	Transient crasher (self-recovers)	Recovers unaided	Recovers (attribution unclear — that's the finding)
W2	Bad current revision, good previous	Never recovers	<code>rollout_undo</code> → recovered
W3	Bad current AND previous revision	Never recovers	Fix applied → verification fails → rolled_back , then <code>exhausted</code>

Day 5 — Deployed-mode verification

Task 6 — Run the full three-workload set in deployed mode (`make docker-build deploy`), not `make run` . Confirm RBAC covers everything the new code touches (the attempt-tracking and evidence-filtering paths may need additional verbs). Fix what breaks.

✓ **Success = permanently-broken Deployments terminate at 3 attempts and go quiet; evidence is frozen and controller-generated events are filtered; three named workloads run reproducibly in deployed mode.**

Ananya — Safety Layer: Durable Audit, Timing Decomposition, Attribution

Mission: your loop works and produced four clean terminal outcomes. Week 3 is about making it *measurable* — a durable audit sink, decomposed timing, and closing the attribution gap you flagged yourself. Resist adding safety features; harden and instrument what exists.

Your Week 2 timings (restart_pod→recovered 65.1s; rolled_back at 20.0s and 30.0s; rollout_undo→recovered 65.1s) are the first real evidence the state machine behaves correctly under both exit paths. Protect that. Everything this week builds on it.

Day 1 — Durable audit sink (blocking all metrics)

Task 1 — JSONL to stdout cannot be read by Subhashini's metrics module. Write the audit log to a **durable, machine-readable sink**: append-only JSONL to a file on a mounted volume, one line per state transition, with a stable schema. Every line needs incidentID, attemptNumber, timestamp, state, action, result, plus the workload/arm label so runs can be grouped later.

Paste to your AI: "My Kubernetes operator writes an append-only JSONL audit log to stdout. I need it durable and readable by a separate metrics process. Show me how to write JSONL to a file on a mounted volume from a controller-runtime manager, with safe concurrent appends, and a stable schema with fields: incidentID, attemptNumber, timestamp, state, action, result, workload, armLabel. Explain the volume choice for a K3s cluster and what happens to the log if the controller pod restarts mid-incident."

Coordinate the schema with Subhashini today, not later. She computes every metric from these lines. If the field names shift after runs begin, the data is unusable.

Day 2 — Timing decomposition (contribution #4)

Task 2 — Emit TTD and TTM as separate decomposed fields. Your 65.1s recovered figure is dominated by the 60s stability window — which means **TTM has a hard 60s floor by construction**. If you report a single TTM number it looks poor next to comparators, and worse, it hides where the time actually goes. Emit four separate durations per attempt:

- t_detect — fault injection → DetectionEvent
- t_classify — classifier call duration (the LLM inference component)
- t_apply — action dispatch → action confirmed applied
- t_verify — verification window duration (constant by design; report it as such)

State the 60s floor explicitly in the report. "Our TTM is bounded below by the stability window we chose" is a design property, not a weakness — but only if you say it first. If an examiner notices it before you mention it, it reads as an oversight.

Day 3 — Close the attribution gap you flagged

Task 3 — You correctly flagged that truncating ActionStartedAt to seconds permits a pre-action pod created in the same second to be accepted as post-action. Fix it exactly rather than with timestamps: **capture the set of pod UIDs owned by the Deployment immediately before the action**, and reject any pod whose UID is in that set. UIDs are exact and need no clock precision.

Paste to your AI: "In Go with controller-runtime, before performing a remediation I want to capture the set of Pod UIDs currently owned by a Deployment (via its ReplicaSets). After the action, my verifier must reject any pod whose UID is in that pre-action set, so I only ever verify against genuinely new pods. Show me the code for both halves and explain why UID comparison is more reliable than comparing creationTimestamp against an action start time truncated to seconds."

If this turns out to be more than a day's work, **stop and document it as a threat to validity instead** — a stated limitation is acceptable; a silent one is not.

Day 4-5 — Support the experiment runs

Task 4 — Be on hand while Subhashini runs the arms. Expect edge cases your unit tests didn't cover: an incident that reaches exhausted mid-verification, the W1 transient workload recovering *during* your window (does that count as recovered-by-us or recovered-unaided?), and back-to-back incidents on the same Deployment across cooldown boundaries.

The W1 question is a real definitional one, decide it before the runs: if the pod self-recovers inside your verification window after your action, your state machine will record recovered — but the recovery may not be yours. This is precisely why the null-action control exists. Make sure the audit line carries enough information to distinguish the two cases later.

✓ **Success = durable JSONL audit on disk with a schema Subhashini can parse; four decomposed timing fields per attempt; UID-based attribution or a written threat-to-validity entry.**

Subhashini — Live Classifier, Metrics, and the Experiment Runs

Mission: your classifier is aligned with plan and the integrated flow is sanity-checked — good. Week 3 is the part that was always yours and was always going to be the hard half: get the *live* LLM into the loop, compute the metrics, and run the experiments that produce every number in the report.

The gap that matters most: the reproducible demo path uses the deterministic mock classifier, not a live LLM. That means you currently have **zero data** on classifier validity, latency or cost — three of the numbers your contributions depend on. Closing this is your Day 1-2.

Day 1-2 — Live LLM in the loop, mock retained

Task 1 — Make the classifier backend switchable (`mock` | `live`) by config, defaulting to mock. You keep the mock for demo stability and determinism; the live path produces your data. Do not replace one with the other.

Task 2 — Instrument every live call: latency, input/output tokens, computed cost, and the raw response. ARBITER reports per-call latency and cost per model; you can match that format cheaply because you are already making the calls.

Paste to your AI: "I have a Go classifier component with a deterministic mock implementation. I want a config-switchable backend (mock or live Claude API) behind the same interface, defaulting to mock. Every live call must record: wall-clock latency, input and output token counts, computed cost, and the raw response, appended to a JSONL file. Show me the interface, both implementations, a timeout with fallback to a deterministic default action, and explain how to keep the demo path fully reproducible while the experiment path uses the live model."

Demo-day safety: the live path needs a timeout and a deterministic fallback. A 4-second hang on stage reads as broken even when it is fine. Pin the model.

Day 2 — Mock-vs-live agreement (a free contribution)

Task 3 — Run the same labelled contexts through both the deterministic mock and the live classifier, and report the **agreement rate**. This directly mirrors ARBITER's covered-context experiment (they reported 30/30 model-context agreement across three models) and it costs you almost nothing since both paths already exist. It answers "does the LLM actually add anything over your rules?" — a question you *will* be asked.

Day 2-3 — Two labelled sets (contribution #3)

Task 4 — Build the adversarial set (~15 cases): off-allowlist actions, non-existent targets, malformed JSON, missing required fields, empty input, out-of-bounds parameters, and semantically inconsistent output (claims "bad rollout" with no recent change event — the semantic guard case).

Task 5 — Build the legitimate set (~30 cases) of valid, in-bounds proposals that **must be accepted**.

Report both sides or the number is meaningless. A validator that rejects everything scores 100% on adversarial rejection. ARBITER reports 11/11 rejected and 77/78 legitimate allowed — match that shape exactly, and always state "measured on N hand-labelled cases" rather than a bare percentage.

Day 3 — Metrics module

Task 6 — Compute, from Ananya's durable audit JSONL, using the frozen definitions from the shared Day 1 doc:

- **Rollback trigger rate** — over attempts (primary) and incidents (secondary)
- **Recovery rate per workload per arm**, and **attributable recovery** = (enabled – disabled) per workload
- **TTD, and TTM decomposed** into detect / classify / apply / verify
- **Outcome distribution** across the five terminal states, including escalation and rejection rates
- **Validator:** adversarial rejection rate + legitimate acceptance rate
- **Classifier:** per-call latency, cost, mock-vs-live agreement

Paste to your AI: "I have an append-only JSONL audit log where each line is a state transition with fields incidentID, attemptNumber, timestamp, state, action, result, workload, armLabel. Write a Python module that groups lines into incidents and attempts and computes: rollback rate over attempts and over incidents; recovery rate per workload per arm; attributable recovery as the difference between enabled and disabled arms; TTD and TTM decomposed into stages; and the distribution across five terminal outcomes. Output a JSON report plus a summary table. Handle incomplete incidents (controller restarted mid-run) explicitly rather than dropping them silently."

Day 4 — The experiment runs (this is the data)

Task 7 — Run the matrix. Three workloads × two conditions × N=5, live classifier throughout:

Arm	Workload	Controller	N	What it measures
A1	W1 transient	Enabled	5	Recovery where K8s could also have recovered
A2	W1 transient	Disabled	5	The nonzero baseline — the whole point
B1	W2 bad revision	Enabled	5	Attributable recovery via rollout_undo
B2	W2 bad revision	Disabled	5	Confirms baseline ≈ 0
C1	W3 unrecoverable	Enabled	5	Rollback fires — contribution #1
C2	W3 unrecoverable	Disabled	3	Confirms baseline = 0 (cheap sanity check)

Budget ~3 hours of cluster time including setup and teardown. Run C1 first — if rollback does not fire there, everything else can wait while it gets fixed. Archive raw audit logs per run; never overwrite.

Day 5 — Grafana and the results table

Task 8 — One Grafana panel showing an incident lifecycle end-to-end (detected → classified → remediating → verifying → recovered/rolled_back) with real data. It does not need to be beautiful; it needs to show the loop live for the demo.

Task 9 — Produce the results table in final form, every cell traceable to a run. Any number that cannot be traced to an archived log does not go in the report.

✓ **Success = live classifier instrumented, both labelled sets scored two-sided, all six arms run and archived, metrics computed from real logs, one live Grafana panel.**

What Week 2 made obsolete or newly risky

Obsolete — stop referring to these

- "Wire detection into the safety layer" and the Week 2 contract-lock items — done, and the four terminal outcomes are observed evidence.
- The "Option A fallback: drop the LLM entirely" escape hatch — the classifier is integrated and aligned; dropping it now would cost more than keeping it.
- The original two-workload test design — superseded by the three-workload set above.

Newly risky — watch these

- **Cross-package coupling.** Week 2 saw four cross-owner behavioural commits. With three people editing shared paths in the final build week, a refactor by one silently breaks another's tests. Rebase daily; run the full suite before pushing.
- **TTM's 60s floor** makes your recovery time look worse than it is against comparators unless decomposed and explained.
- **Mock-vs-live divergence.** Every Week 2 result was produced with the mock. If the live classifier behaves differently, some Week 2 conclusions may not hold — find out on Day 2, not Day 5.
- **The verifier constants (30s / 60s) are undefended defaults.** You cannot tune them this week. Say plainly in the report that they were chosen a priori and not tuned against data, and note that no paper in your corpus evaluates stability-window sensitivity either — that is honest and it doubles as future work.

- **Evidence freezing loses late-emerging causes.** A stated tradeoff, not a hidden one.

End-of-Week-3 definition of done. Every number in your results table exists, comes from an archived run, and is traceable. Specifically: rollback fires and is counted over a defined population; recovery is reported net of a measured baseline that is **nonzero for at least one workload**; the validator is scored on both adversarial and legitimate inputs; TTM is decomposed. If those four exist by Friday, Week 4 is writing and rehearsal. If they do not, Week 4 becomes a scramble — so protect the Day 4 run block and let nothing else consume it.